

Cours 2

1. Variables structurées
 - Tableaux statiques et dynamiques
 - Structures
2. Structures de données en Python
 - Listes en Python

Pourquoi différentes structures de données ?

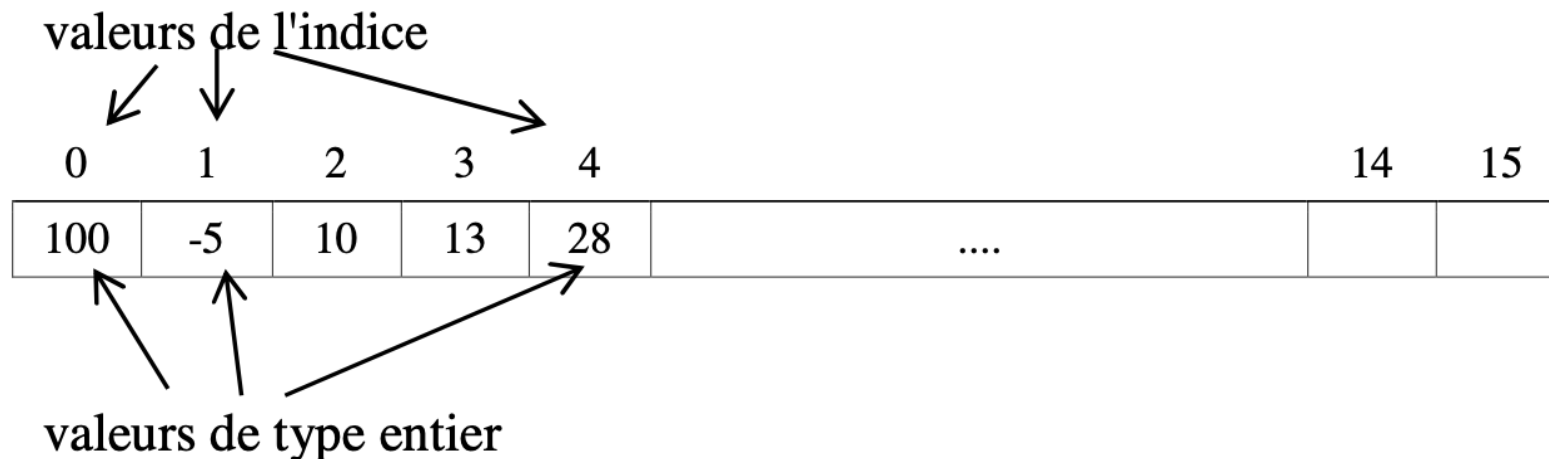
- Les structures de données sont des moyens spécifiques d'organiser et de stocker des données afin qu'elles puissent être consultées et traitées de manière efficace.
 - Les structures de données définissent la relation entre les données et les opérations pouvant être effectuées dessus.
- La spécialisation des structures de données rend la programmation plus simple et plus efficace (complexité).
 - Une structure de données inappropriée peut entraîner des durées de fonctionnement (inutilement) longues, des pertes de mémoire et un gaspillage de stockage. Ce qui peut raisonnablement entraîner une perte de plusieurs millions d'euros, voire de quelques jours quand on parle de données à l'échelle.

Nécessité des variables structurées

- Les **variables simples** peuvent avoir qu'une valeur à un instant donné. Si une nouvelle valeur leur est affectée, la précédente est perdue.
- **Variables de type tableau** permettent de regrouper une série d'éléments de même type. Chaque élément du tableau est accessible directement par son numéro d'ordre.
 - Exemple : tableau de jours de la semaine – 'Lundi', 'Mardi', ..., 'Dimanche'
- **Variables de type structure** permettent matérialiser un lien entre des données qui caractérisent une même entité dans une seule variable.
 - Exemple : les caractéristiques d'une date – jour, mois, année.
- Caractéristique commune des variables structurées : elles peuvent avoir plusieurs valeurs à un instant donné.
 - Mais elles se définissent et s'utilisent de manières différentes.

Tableaux

- Un **tableau** à une dimension est une variable structurée qui peut contenir une série d'éléments de même type. Un tableau est caractérisé par un **identificateur**.
- Chaque élément du tableau est repéré par un **indice** (un nombre entier) qui prend ses valeurs entre deux bornes (bornes de début et de fin).
- Certains éléments de tableau peuvent être vides à un instant donné.



Déclaration d'un tableau

- **Syntaxe :**

```
<identificateur> : tableau [<indice_début> ... <indice_fin>] de <type>
```

- Les informations nécessaires pour la déclaration d'un tableau :
 - les **bornes** : la borne inférieure et la borne supérieure que peut prendre son indice, pour déterminer la réservation d'un certain nombre d'éléments pour le tableau
 - le **type** de valeurs prises par les éléments d'un tableau
- **Exemple :**

```
table1 : tableau [0 .. 15] d'entiers
```

Mémoire alloué pour un tableau

- Un tableau est par nature une variable **statique** (en algorithmique), à laquelle on donne une taille qui ne varie pas.
- Le contexte du programme doit nous permettre de déterminer la taille nécessaire.
- Un tableau peut avoir été déclaré avec un intervalle d'indice de 1 à 100 et ne contenir que 20 valeurs
 - Place déclarée est de 100 éléments
 - Place occupée est 20 éléments
- Il ne faut pas surdimensionner un tableau pour ne pas gaspiller la place en mémoire vive !
- Si le problème nécessite de faire évoluer la taille, un tableau n'est pas le type de variable approprié. Il faut alors utiliser **des variables dynamiques**.

Accès à un élément de tableau

- Chaque élément de tableau est repéré par une valeur de l'indice.
- **Syntaxe :**

```
<identificateur>[<indice>]
```

- L'indice peut être :
 - une constante : `table1[2]`
 - une variable : `ind ← 3 ; table1[ind]`
 - une expression : `table1[ind + 1]`

Élément d'un tableau

- Un élément de tableau se comporte exactement comme une variable simple.
- Il peut faire partie de n'importe quelle expression.
- On peut utiliser un élément de tableau dans les instructions suivantes :
 - dans une instruction de lecture : `lire(tab[4])`
 - dans une instruction d'écriture : `écrire(tab[4])`
 - dans une instruction d'affectation : `tab[5] ← 45`
 - dans une expression : `écrire(tab[5] * 10)`
 - à la place d'un paramètre effectif d'une fonction : `échanger(tab[1], tab[2])`

Algorithme pour remplir un tableau : nombre connu de valeurs

Algorithme remplir_tableau_1

texte : tableau[0..99] de caractères /* le texte */

ncar : entier /* nombre de caractères */

i : entier /* variable de boucle et indice */

Début

lire (ncar)

pour i de 0 à ncar-1 faire

lire(texte[i])

finpour

Fin

17	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17			
ncar	i	l		é	t	a	i	t		u	n	e		f	o	i	s			
	texte																			

Exercices 1-2 :

Algorithmes pour remplir un tableau

Modifiez l'algorithme précédent :

1. Proposez un algorithme de remplissage d'un tableau quand le dernier élément est reconnu et rangé aussi dans le tableau.
2. Proposez un algorithme de remplissage d'un tableau quand le dernier élément est reconnu, mais fictif et donc non rangé dans le tableau.

Exercice 1 : solution

Algorithme remplir_tableau_2

texte : tableau[0..99] de caractères /* le texte */

ncar : entier /* nombre de caractères */

Début

ncar $\leftarrow$ 0

répéter

lire(texte[ncar])

ncar $\leftarrow$ ncar + 1

jusqu'à texte[ncar] = '.'

Fin

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18		
18	i	l		é	t	a	i	t		u	n	e		f	o	i	s	.		
ncar	texte																			

Exercice 2 : solution

Algorithme remplir_tableau_2

texte : tableau[0..99] de caractères

/* le texte */

ncar : entier

/* nombre de caractères */

carlu : caractère

/* variable de saisie */

Début

ncar $\leftarrow$ 0

lire(carlu)

tantque carlu $\neq$ '#' faire

 texte[ncar] $\leftarrow$ carlu

 lire(carlu)

 ncar $\leftarrow$ ncar + 1

fintantque

Fin

18	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	#
ncar	i	l		é	t	a	i	t		u	n	e		f	o	i	s	carlu
	texte																	

Algorithmes pour parcourir et exploiter un tableau

- **Problème** : remplacer les 'i' par des 'x' dans un tableau avec le nombre d'éléments connus.

Algorithme parcourir_tableau_1

```
    texte : tableau[0..99] de caractères    /* le texte */  
    ncar : entier                          /* nombre de caractères */  
    k : entier                             /* variable de boucle */
```

Début

```
    lire(ncar)  
    pour k de 0 à ncar faire  
        si texte[k] = 'i' alors  
            texte[k] ← 'x'  
        finsi  
    finpour
```

Fin

Exercice 3 :

- Proposez un algorithme de recherche d'un élément dans un tableau. Le dernier élément du tableau est '.'

Tableaux à plusieurs dimensions

- Pour déclarer un tableau à n dimensions, il faut donner n intervalles de valeurs pour les indices, ainsi que le type des valeurs rangées dans le tableau.
- Syntaxe :

```
<identificateur> : tableau [ <a1... b1>, ..., <an..bn> ] de <type>
```

- L'accès se fait avec les valeurs des n indices :

```
<identificateur>[<valeur i1>, ..., <valeur in>]
```

Exemple de tableau à deux dimensions

- **Déclaration** d'un tableau à deux dimensions

mat : tableau[0..3, 0..5] de caractères

	0	1	2	3	4	5
0	A	C	J	H	R	U
1	E	B	P	I	S	H
2	O	D	K	Q	T	G
3	N	M	L			

- **Accès** à l'élément .

mat[2, 3] = 'Q'

Tableau dynamique (1/2)

- Les tableaux définis jusqu'ici sont dits **statistiques**, car il faut qu'au moment de l'écriture d'algorithme, on a la taille maximale que pourra atteindre le tableau.
- Dans certaines situations, on ne peut pas savoir la taille du tableau dans la phase de définition d'un algorithme (programmation).
- Les **tableaux dynamiques** sont des tableaux dont la taille n'est définie que lors de l'exécution.
- Pour créer un tableau dynamique, il suffit de lui affecter **une taille vide**.

Tableau dynamique (2/2)

- Syntaxe :

```
<identificateurTableauDyn> : tableau[] de <type>
```

- Comme un tableau dynamique ne possède pas de taille prédéfinie, il convient de redimensionner le tableau avant de pouvoir s'en servir.
- Syntaxe :

```
Redimensionner <identificateurTableauDyn>[N]
```

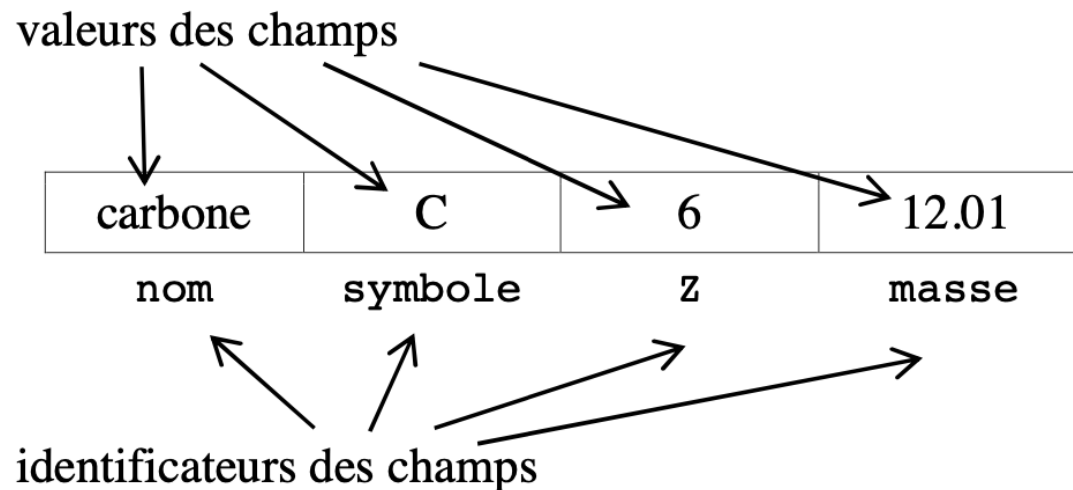
- Exemple :

```
t : tableau[] d'entiers
```

```
Redimensionner t[11]
```

Structures

- Structure est un autre type de variable structurée.
- Structure peut contenir **plusieurs valeurs de types différents**. Elle est caractérisée par un **identificateur**.
- Chaque valeur est rangée dans un champ et repérée par un **identificateur de champ**.



Déclaration d'une structure

- Syntaxe :

```
<identificateur> : structure
                                <identificateur champ 1> : <type>
                                ...
                                <identificateur champ n-1> : <type>
                                <identificateur champ n> : <type>
                                fin
```

- Exemple :

```
element : structure
            nom : chaîne de caractères           /* nom de l'élément */
            symbole : chaîne de caractères /* symbole chimique */
            Z : entier                           /* numéro atomique */
            masse : réel                         /* masse atomique */
            finstructure
```

Accès aux champs

- Chaque champ d'une structure est accessible via le nom d'un champ séparée d'un identificateur d'une variable de type structure par le point.

- **Syntaxe :**

```
<identificateur>.<identificateur champ>
```

- On peut utiliser un champ d'une structure dans les instructions suivantes
 - dans une instruction de lecture : `lire(element.symbole)`
 - dans une instruction d'écriture : `écrire(element.symbole)`
 - dans une instruction d'affectation : `element.symbole ← 'Na'`
 - dans une expression : `m ← element.masse * 4`
 - à la place d'un paramètre réel dans un appel de fonction : `échanger(element1.Z, element2.Z)`

Définition de types

- Il peut être pratique de définir des types et les utiliser dans des déclarations de variables.

- **Syntaxe :**

```
<type> = <définition du type>
```

- **Exemples :**

Définition de type :

```
typeTab = tableau [0..15] d'entiers
```

Déclaration de variable :

```
tab : typeTab
```

Définition de type

```
typeEl = structure
```

```
  nom : chaîne de caractères
```

```
  symbole : chaîne de caractères
```

```
  Z : entier
```

```
  masse : réel
```

```
finstructure
```

Déclaration de variable :

```
elt : typeEl
```

Types complexes

- On peut construire des types complexes en composant tableaux et structures :
 - un élément de tableau peut être une structure,
 - un champ de structure peut être un tableau
- Il n'y a pas de limite à la composition.

Exemple : tableaux de structures(1/2)

- Exemple : table des éléments chimiques

	Nom	symbole	Z	masse
0	Hydrogène	H	1	1
1	Hélium	He	2	4
2	Lithium	Li	3	6,9
3	Béryllium	Be	4	9
4	Bore	B	5	10,8
5	Carbone	C	6	12,01

Exemple : tableaux de structures (2/2)

- Définitions des types :

typeEl = structure

nom : chaîne de caractères

symbole : chaîne de caractères

Z : entier

masse : réel

finstructure

typeTab = tableau[0..120] de typeEl

- Déclaration :

table_periodique : typeTab

- Accès à un élément :

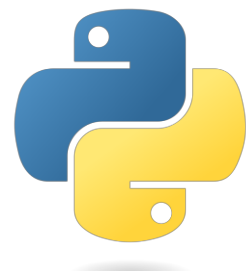
table[4].masse contient la valeur 10,8

table[2].symbole contient la valeur 'Li'

	Nom	symbole	Z	masse
0	Hydrogène	H	1	1
1	Hélium	He	2	4
2	Lithium	Li	3	6,9
3	Béryllium	Be	4	9
4	Bore	B	5	10,8
5	Carbone	C	6	12,01

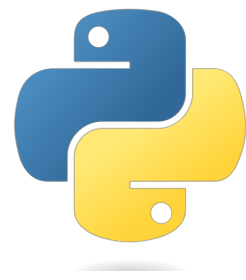
Conclusion sur les variables structurées

- Les tableaux et les structures permettent de composer des structures complexes dans lesquelles peuvent être organisés des rangements des données qu'on appelle structures de données.
- Les structures de données et les méthodes algorithmiques qui les exploitent sont le plus souvent dépendantes les unes des autres.



Définition d'une liste

- Une **liste** est une collection ordonnée de valeurs.
- Dans une liste, chaque valeur occupe une position bien définie que l'on repère par un entier appelé indice.
- La **première valeur** est associée à l'**indice 0**, la seconde à l'**indice 1**, etc.
- Une liste a une longueur (i.e. un nombre d'éléments) **finie**
 - La liste vide a pour longueur 0.
- En Python, il n'est pas nécessaire que tous les éléments d'une liste soient du même type.
- Plus d'info : <https://docs.python.org/3/tutorial/datastructures.html>



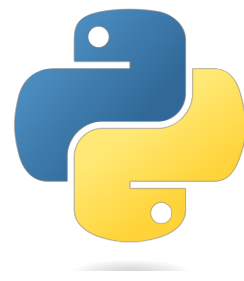
Type list de Python

- Le type `list` de Python est implémenté à l'aide de tableaux dynamiques
 - la suppression ou l'ajout ailleurs qu'en fin de liste nécessite de décaler les valeurs de fin de liste, et n'est donc pas réalisé en temps constant.
 - l'accès à un élément quelconque est réalisé en temps constant, ce qui n'est pas le cas avec une liste chaînée.

Type Python	Type abstrait	Opération	Exemple	Complexité
<code>list</code>	Tableau dynamique	Ajout à la fin	<code>lst.append(x)</code>	$O(1)$ ★
<code>lst=[]</code>		Accès à un élément	<code>lst[i]</code>	$O(1)$
<code>len(lst)</code>		Modification d'un élément	<code>lst[i] = x</code>	$O(1)$
		Effacement d'un élément	<code>del lst[i]</code>	$O(n)$
		Insertion d'un élément	<code>lst.insert(i, x)</code>	$O(n)$
		Recherche d'un élément	<code>x in lst</code>	$O(n)$

Complexité d'algorithme :

Réaliser un calcul de complexité en temps revient à compter le nombre d'**opérations élémentaires** (affectation, calcul arithmétique ou logique, comparaison...) effectuées par l'algorithme.



Création de liste

- Pour créer une liste contenant des éléments définis :

```
l1 = [1, 5, 7]
```

- Pour créer une liste vide (ne contenant aucun élément) :

```
l2 = []
```

- La fonction `len` retourne la taille d'une liste passée en argument :

```
print(len(l1)) ⇒ 3
```

```
print(len(l2)) ⇒ 0
```

- La fonction `range(a, b [, c])` retourne une séquence contenant les entiers de **a** (inclus) à **b** (exclu) :

```
l3 = range(1, 10) ⇒ (1, 2, 3, ..., 9)
```

```
l4 = range(10) ⇒ (0, 1, 2, ..., 9)
```

```
l5 = range(0,10,2) ⇒ (0,2,4,...,8)
```



Accès aux éléments d'une liste (1/2)

- Pour accéder au i -ème élément d'une liste

`l[i]`

- Le **premier indice** d'une liste est **0**

`l = [1, 5, 7]`

`print(l[1])` $\Rightarrow$ 5

- On peut accéder au dernier élément d'une liste en demandant l'élément d'indice **-1** :

`print(l[-1])` $\Rightarrow$ 7

- Pour une liste de taille n , les valeurs d'indice valides sont les entiers compris entre $-n$ et $n - 1$ (inclus).



Accès aux éléments d'une liste (2/2)

- On peut accéder à une sous-partie d'une liste à l'aide de la syntaxe

`l[d:f]`

où **d** est l'indice de début et **f** est l'indice de fin (exclu).

- Exemple :

```
l2 = [1, 5, 7, 8, 0, 9, 8]
```

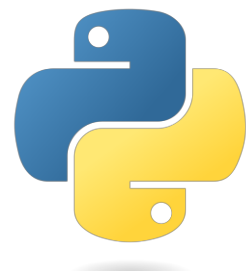
```
print(l2[2:4]) ⇒ [7, 8]
```

- Si on omet l'indice de début, la sélection commence au début de la liste :

```
print(l[:3]) ⇒ [1, 5, 7]
```

- Si on omet l'indice de fin, elle s'étend jusqu'à la fin de la liste :

```
print(l[5:]) ⇒ [9, 8]
```



Parcours d'une liste

- Lorsque l'on parcourt une liste, on peut vouloir accéder :
 - aux éléments stockés dans la liste uniquement ;
 - aux indices de la liste uniquement (même si c'est rare) ;
 - aux indices de la listes et aux éléments associés.



Parcours des éléments d'une liste

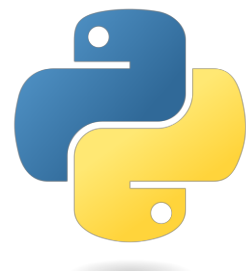
- Pour parcourir les éléments d'une liste, on utilise une boucle **for**

```
l = [1, 5, 7]
```

```
for elem in l:
```

```
    print(elem)
```

- La variable **elem** va prendre successivement pour valeur chacun des éléments de la liste.



Parcours d'une liste par indices

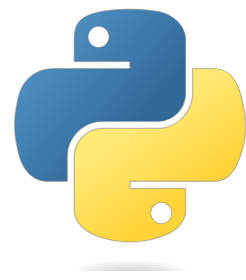
- On sait que les indices d'une liste sont les entiers compris entre 0 (inclus) et la taille de la liste (exclu).
- On va utiliser la fonction `range` et une boucle `for`

```
l = [1, 5, 7]
```

```
n=len(l)
```

```
for i in range(n):
```

```
    print(i, l[i])
```



Parcours d'une liste par éléments et indices

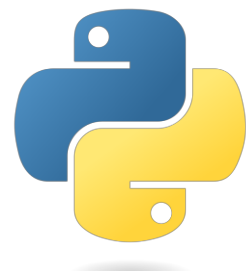
- Si on a besoin de manipuler simultanément les indices d'une liste et les éléments associés, on utilise la fonction `enumerate` :

```
l = [1, 5, 7]
```

```
for i, elem in enumerate(l):
```

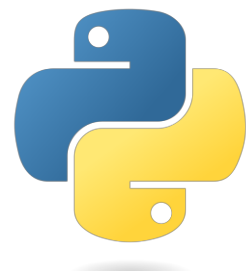
```
    print(i, elem)
```

- A chaque itération d'une boucle `for`, on met à jour les variables
 - `i` – qui contient l'indice courant
 - `elem` – qui contient l'élément se trouvant à l'indice `i` dans la liste `liste`.



Exercice 4

- Écrivez une fonction en Python qui permette de calculer l'argmax d'une liste.
- La fonction doit retourner l'indice auquel est stockée la valeur maximale de la liste.
- Si cette valeur maximale est présente plusieurs fois dans la liste, on retournera l'indice de sa première occurrence.



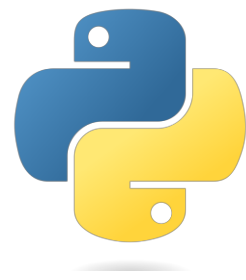
Insertion d'élément d'une liste

- Pour insérer un nouvel élément dans une liste, on peut :
 - rajouter un élément à la fin de la liste à l'aide de la méthode `append` ;
 - insérer un élément à l'indice `i` de la liste à l'aide de la méthode `insert`.

- **Exemple :**

```
l = [1, 5, 7]
l.append(2)
print(l) ⇒ [1, 5, 7, 2]
```

```
l.insert(2, 0) # insère la valeur 0 à l'indice 2
print(l) ⇒ [1, 5, 0, 7, 2]
```



Suppression d'élément d'une liste (1/2)

- Pour supprimer un élément d'une liste, on peut :
 - supprimer l'élément situé à l'indice *i* dans la liste, à l'aide de la méthode **pop** ;
 - supprimer la première occurrence d'une valeur donnée dans la liste à l'aide de la méthode **remove**.

- **Exemple :**

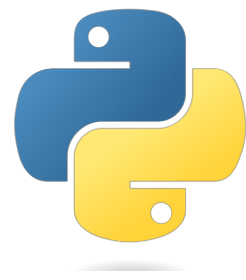
```
l = [1, 5, 7]
```

```
l.pop(1) # supprimer l'élément d'indice 1
```

```
print(l) ⇒ [1, 7]
```

```
l.pop() # par défaut, supprime le dernier élément de la liste
```

```
print(l) ⇒ [1]
```



Suppression d'élément d'une liste (2/2)

- Pour supprimer un élément d'une liste, on peut :
 - supprimer l'élément situé à l'indice `i` dans la liste, à l'aide de la méthode `pop` ;
 - supprimer la première occurrence d'une valeur donnée dans la liste à l'aide de la méthode `remove`.

- **Exemple :**

```
l = [7, 1, 5, 1]
```

```
l.remove(1) # supprime la première occurrence de 1
```

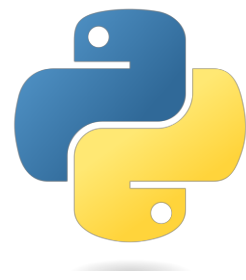
```
print(l) ⇒ [7, 5, 1]
```

- La méthode `pop` retourne la valeur supprimée

```
l = [1, 5, 7]
```

```
v = l.pop(1)
```

```
print(v) ⇒ 5
```



Recherche d'élément dans la liste

- Pour trouver l'indice de la première occurrence d'une valeur dans une liste, on utilisera la méthode **index**

```
l = [1, 5, 7]
```

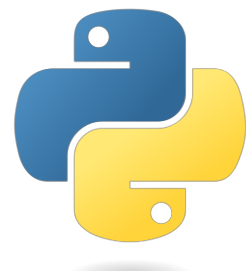
```
print(l.index(7)) ⇒ 2
```

- Si on veut simplement savoir si une valeur est présente dans la liste, on peut utiliser le mot-clé **in**

```
l = [1, 5, 7]
```

```
if 5 in l:
```

```
    print("5 est dans liste")
```



Création de listes composites

- On peut concaténer deux listes à l'aide de l'opérateur +

```
l1 = [1, 5, 7]
```

```
l2 = [3, 4]
```

```
l = l1 + l2
```

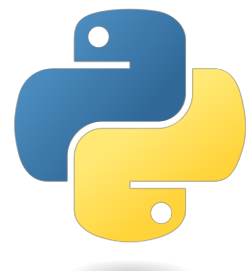
```
print(l) ⇒ [1, 5, 7, 3, 4]
```

- L'opérateur * peut aussi être utilisé pour des listes

```
l1 = [1, 5]
```

```
l2 = 3 * l1
```

```
print(l2) ⇒ [1, 5, 1, 5, 1, 5]
```



Tri de liste

- On peut trier les éléments contenus dans une liste à l'aide de la fonction **sorted**

```
l = [4, 5, 2]
```

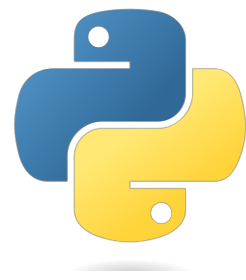
```
l2 = sorted(l)
```

```
print(l2) ⇒ [2, 4, 5]
```

- On peut aussi trier des listes de chaînes de caractères

```
l = ["a", "z", "c"]
```

```
print(sorted(l)) ⇒ ['a', 'c', 'z']
```



Copie de liste (1/2)

- La copie ne pose pas de problème pour la plupart des variables :

```
a = 12
```

```
b = a
```

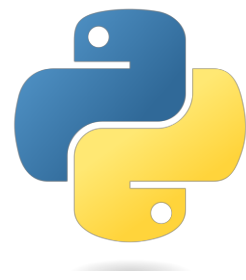
```
a = 5
```

```
print(a, b)
```

- Cela ne se passe pas de la même façon pour les listes. Si on écrit :

```
l2 = l
```

- On ne recopie pas le contenu de `l` dans `l2`.
- On crée une variable `l2` qui va “pointer” vers la même position dans la mémoire de votre ordinateur que `l`.



Copie de liste (2/2)

- Si on modifie `l` après l'instruction `l2 = l`, la modification sera répercutée sur `l2`:

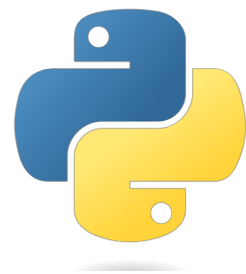
```
l = [1, 5, 7]
```

```
l2 = l
```

```
l[1] = 2
```

```
print(l) ⇒ [1, 2, 7]
```

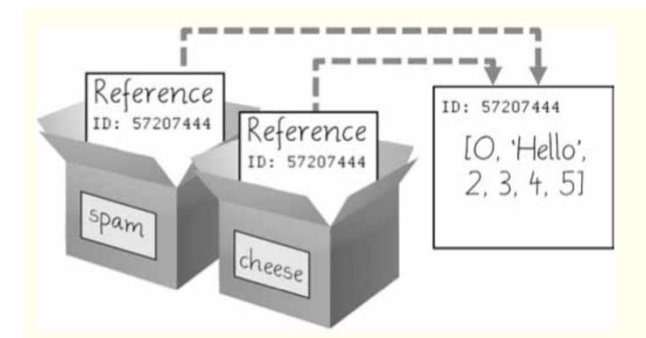
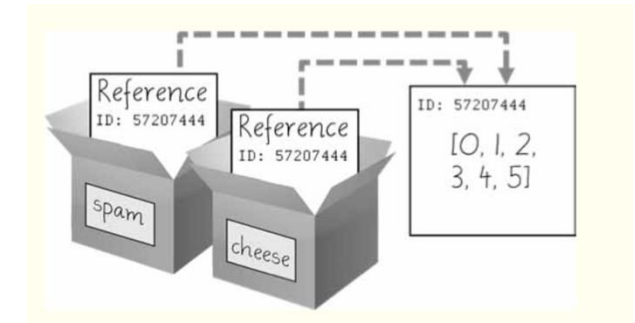
```
print(l2) ⇒ [1, 2, 7]
```

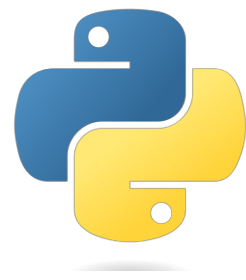


Référence

```
>>> spam = [0, 1, 2, 3, 4, 5]
>>> cheese = spam # La référence est copiée
>>> cheese[1] = 'Hello!' # Changement de la valeur de la liste
```

```
>>> spam
[0, 'Hello!', 2, 3, 4, 5]
>>> cheese # La liste cheese référence à la même liste
[0, 'Hello!', 2, 3, 4, 5]
```





Identité et la fonction id() (1/2)

```
>>> bacon = 'Hello'
```

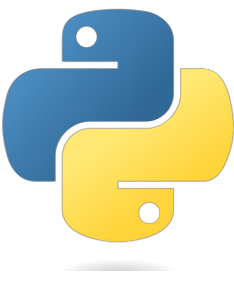
```
>>> id(bacon)
```

```
44491136
```

```
>>> bacon += ' world!' # Nouvelle chaine de caractère est créée
```

```
>>> id(bacon) # bacon référence à une autre chaine de caractère
```

```
44609712
```



Identité et la fonction id() (2/2)

```
>>> eggs = ['cat', 'dog'] # Création d'une liste
```

```
>>> id(eggs)
```

```
35152584
```

```
>>> eggs.append('moose') # Modification de la liste
```

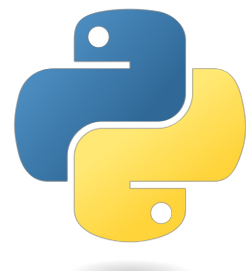
```
>>> id(eggs) # référence est la même
```

```
35152584
```

```
>>> eggs = ['bat', 'rat', 'cow'] # Création de la nouvelle liste
```

```
>>> id(eggs) # référence est complètement différente
```

```
44409800
```



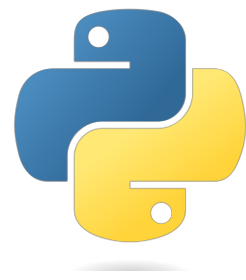
Copie de liste (2/3)

- Si on modifie `l` après l'instruction `l2 = l`, la modification sera répercutée sur `l2` :

```
l = [1, 5, 7]
l2 = l
l[1] = 2
print(l) ⇒ [1, 2, 7]
print(l2) ⇒ [1, 2, 7]
```

- Pour éviter ce comportement, il faut effectuer une copie explicite de liste, à l'aide de la fonction `list` :

```
l = [1, 5, 7]
l2 = list(l)
l[1] = 2
print(l) ⇒ [1, 2, 7]
print(l2) ⇒ [1, 5, 7]
```



Copie de liste (3/3)

- Utilisez la méthode `copy()` (ou la méthode `deepcopy()`) :

```
liste = [1, 2, 3]
```

```
liste2 = liste
```

```
liste3 = liste.copy()
```

```
liste = [1,2,3]
print("Liste:", id(liste))

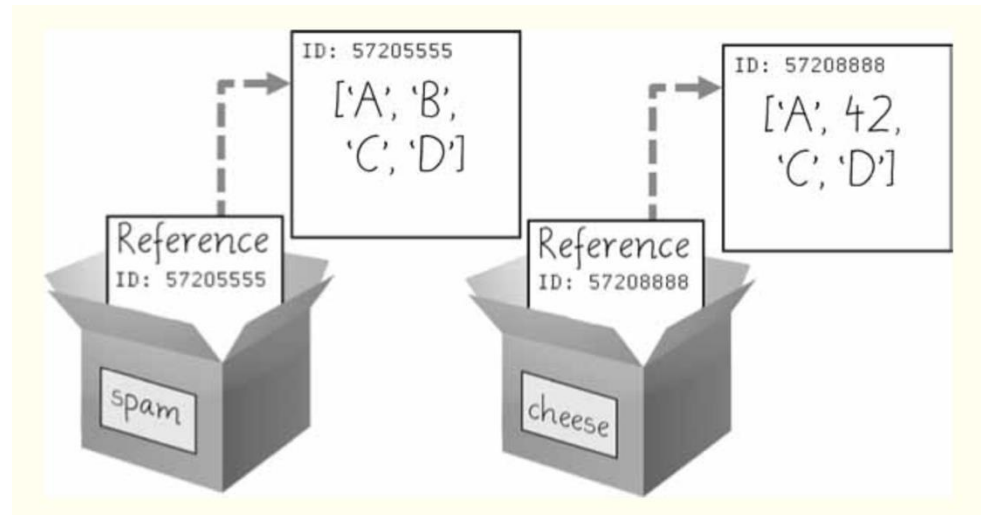
liste2 = liste
print("Liste2:", id(liste2))

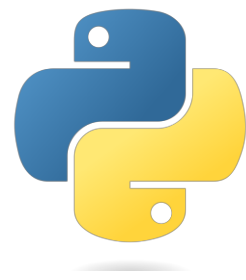
liste3 = liste.copy()
print("Liste3:", id(liste3))
```

Liste: 140207372596672

Liste2: 140207372596672

Liste3: 140207372044416





Parcours simultané de plusieurs listes

- Si on veut afficher les noms et prénoms, on peut effectuer un parcours par indice :

```
preoms = ["Jeanne", "Anne", "Camille"]
```

```
noms = ["Papin", "Bakayoko", "Drogba"]
```

```
for i in range(len(preoms)):
```

```
    print(preoms[i], noms[i])
```

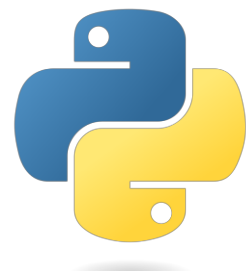
- Si on a besoin de parcourir simultanément plusieurs listes, on utilise la fonction **zip()**

```
preoms = ["Jeanne", "Anne", "Camille"]
```

```
noms = ["Papin", "Bakayoko", "Drogba"]
```

```
for p, n in zip(preoms, noms):
```

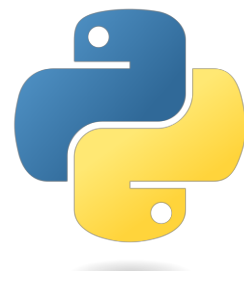
```
    print(p, n)
```



Numpy.array

- Le type tableau `numpy.array` n'est pas le plus courant en Python.
- Il est plus proche des types tableaux disponibles dans d'autres langages de programmation.
- Pour utiliser ce type de tableaux en Python, il est nécessaire :
 - d'installer le module `Numpy`,
 - d'inclure la ligne suivante dans tous les programmes utilisant les tableaux :

```
from numpy import *
```



Déclaration d'un tableau

- Un tableau peut-être déclaré de plusieurs manières :

`from numpy import *`

`array([5,3,2,1,1])` # crée un tableau à partir de ses éléments

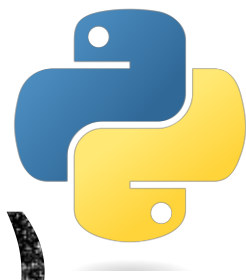
-> `array([5, 3, 2, 1, 1])`

`zeros(3, float)` # crée un tableau rempli de 0

-> `array([ 0., 0., 0.])`

`empty(4, int)` # crée un tableau non initialisé

-> `array([ 0, 8826784, 31983376, 0])`



Opérations sur les tableaux (1/2)

- Longueur de tableau

```
a = array([5,3,2,1,1])
```

```
len(a) # longueur
```

- ou

```
a.size # autre manière d'obtenir la longueur d'un tableau
```

- Premier élément

```
a[0] # premier élément
```

- Sous-tableau

```
a[:3] # sous-tableau correspondant aux 3 premiers éléments
```

```
a[3:] # sous-tableau correspondant aux éléments à partir du 4ème
```



Opérations sur les tableaux (2/2)

- Modifier un élément d'un tableau (identifié par son indice) en utilisant une affectation :

```
a = array([5,3,2,1,1])
```

```
a[1] = 9 # modification du 2ème élément du tableau
```

```
a -> array([5, 9, 2, 1, 1])
```